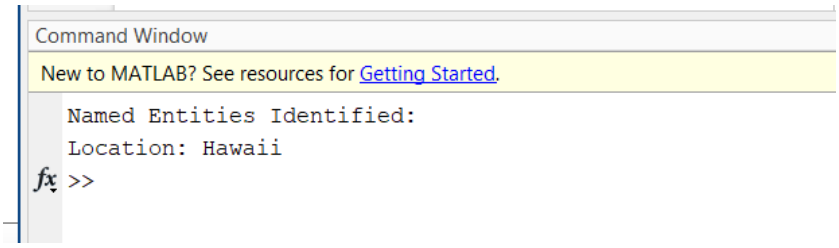
Artificial Intelligent
LAB -7
Natural Language Processing (NLP- Part B)

Code 1:

Named Entity Recognition (NER) Simulation in MATLAB

```
clc; clear;

% Sample input sentence
text = "Barack Obama was born in Hawaii and served as the 44th President of the United States.";

% Define simple rule-based dictionaries
personList = ["Barack Obama"];
locationList = ["Hawaii", "United States"];
organizationList = ["United Nations", "Google", "Microsoft"];

% Tokenize
tokens = split(text);
disp("Named Entities Identified:");

for i = 1:length(tokens)
    word = erase(tokens{i}, '.'); % Remove punctuation
    if any(strcmp(word, personList))
        fprintf("Person: %s\n", word);
    elseif any(strcmp(word, locationList))
        fprintf("Location: %s\n", word);
    elseif any(strcmp(word, organizationList))
        fprintf("Organization: %s\n", word);
    end
end
```

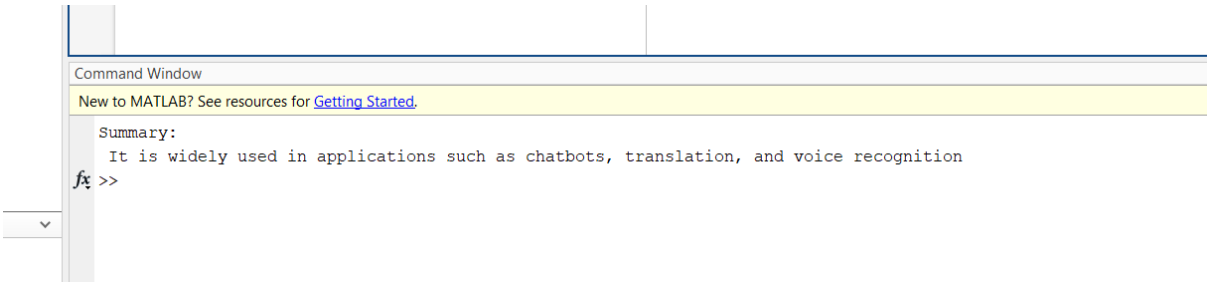


Code: 2

Text Summarization using Keyword Frequency

```
clc; clear;

text = "Natural language processing is a subfield of artificial intelligence. It helps computers understand human language. It is widely used in applications such as chatbots, translation, and voice recognition.";
```

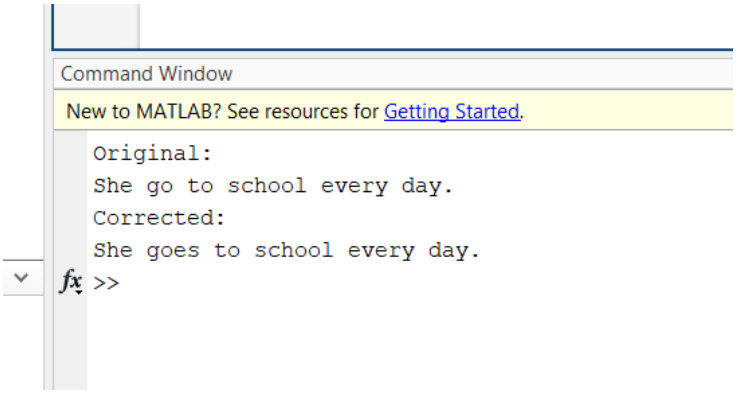
```
% Split into sentences
sentences = split(text, '.');

% Remove empty entries
sentences = sentences(~cellfun('isempty',sentences));

% Tokenize and compute word frequency
allWords = [];
for i = 1:length(sentences)
    tokens = lower(split(sentences{i}));
    allWords = [allWords; tokens];
end

% Count word frequencies
uniqueWords = unique(allWords);
freq = zeros(size(uniqueWords));
for i = 1:length(uniqueWords)
    freq(i) = sum(strcmp(uniqueWords{i}, allWords));
end

% Score sentences by frequency of their words
scores = zeros(length(sentences), 1);
for i = 1:length(sentences)
    tokens = lower(split(sentences{i}));
    for j = 1:length(tokens)
        idx = find(strcmp(tokens{j}, uniqueWords));
        if ~isempty(idx)
            scores(i) = scores(i) + freq(idx);
        end
    end
end

% Pick sentence with highest score as summary
[~, bestIdx] = max(scores);
summary = sentences{bestIdx};

disp("Summary:");
disp(summary);
```



Code: 3

Grammar Correction (Simulation)

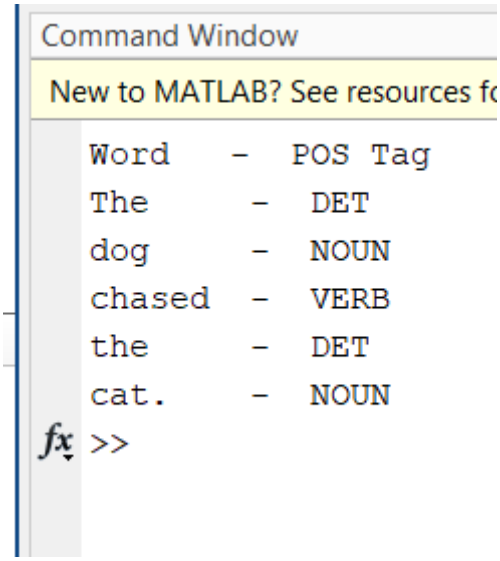
```
clc; clear;

inputSentence = "She go to school every day.";

% Define correction rules
patterns = ["go to school"];
corrections = ["goes to school"];

% Apply correction
corrected = inputSentence;
for i = 1:length(patterns)
    corrected = strrep(corrected, patterns(i), corrections(i));
end

disp("Original:");
disp(inputSentence);
disp("Corrected:");
disp(corrected);
```



Code: 4

Dependency Parsing Simulation (POS tagging-style)

```
clc; clear;

sentence = "The dog chased the cat.";

% Simulated POS tags
words = split(sentence);
tags = ["DET", "NOUN", "VERB", "DET", "NOUN"];

disp("Word   -  POS Tag");
for i = 1:length(words)
    fprintf("%-7s -  %s\n", words{i}, tags{i});
end
```

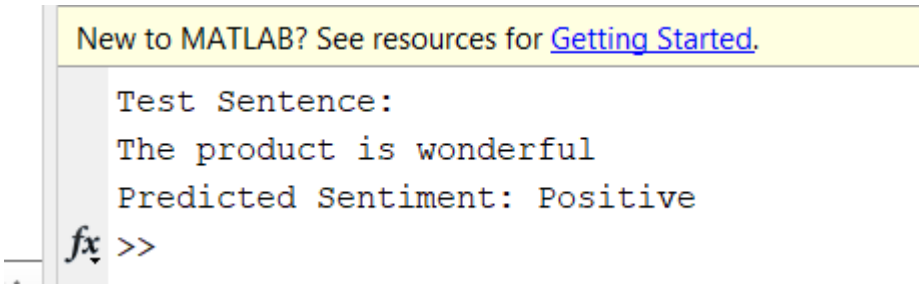


Code: 5

Sentiment Classification using Text + Naive Bayes (Simplified)

```
clc;
clear;

% --------------------------------
% 🐞 Step 1: Example Data
% --------------------------------

% Sample text data and their labels (1 = Positive, 0 = Negative)
texts = [
    "I love this product", ...
    "This is terrible", ...
    "Very happy with the results", ...
    "It is bad and disappointing", ...
    "Absolutely wonderful experience"
];

labels = [1; 0; 1; 0; 1];  % Sentiment labels

% Define a manual bag-of-words (vocabulary)
bag = ["love", "terrible", "happy", "bad", "wonderful"];

% --------------------------------
% 🛠️ Step 2: Text to Binary Feature Matrix
% --------------------------------

% Initialize feature matrix
features = zeros(length(texts), length(bag));

% Fill in binary features for each text
for i = 1:length(texts)
    for j = 1:length(bag)
```

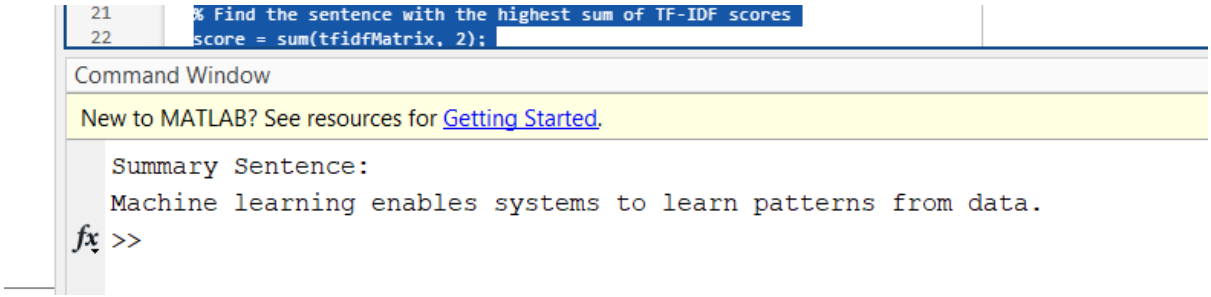
```
        if contains(lower(texts(i)), bag(j))  % use (i) not {i} for string array
            features(i, j) = 1;
        end
    end
end

% Optional: Remove any constant features (to avoid Naive Bayes errors)
features = features(:, var(features) > 0);

% --------------------------------
% 🤖 Step 3: Train Naive Bayes Classifier
% --------------------------------

Mdl = fitcnb(features, labels, 'DistributionNames', 'kernel');  % Use kernel to avoid distribution issues

% --------------------------------
% 🧪 Step 4: Test on New Sentence
% --------------------------------

testText = "The product is wonderful";

% Convert test text into feature vector
testFeat = zeros(1, length(bag));  % 1 row, N features
for j = 1:length(bag)
    if contains(lower(testText), bag(j))
        testFeat(1, j) = 1;
    end
end

% Predict sentiment
pred = predict(Mdl, testFeat);

% Display results
disp("Test Sentence:");
disp(testText);
if pred == 1
    disp("Predicted Sentiment: Positive");
else
    disp("Predicted Sentiment: Negative");
end
```



Code: 6

Text Summarization using TF-IDF

```
% Clear workspace
clear; clc;

% Multiple sentences as paragraph
text = [
    "Artificial Intelligence is transforming every industry.", ...
    "Machine learning enables systems to learn patterns from data.", ...
    "Deep learning uses neural networks with many layers.", ...
    "AI helps businesses automate tasks and improve efficiency."
];

% Convert to tokenized documents
documents = tokenizedDocument(text);

% Create bag-of-words model
bag = bagOfWords(documents);

% Convert to TF-IDF
tfidfMatrix = tfidf(bag);

% Find the sentence with the highest sum of TF-IDF scores
score = sum(tfidfMatrix, 2);
[~, idx] = max(score);

% Display summary
disp("Summary Sentence:");
disp(text(idx));
```

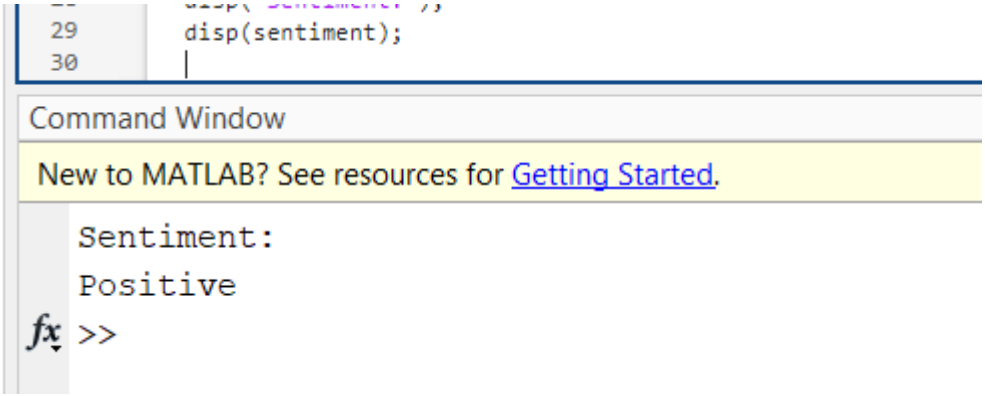


Code: 7

Rule-Based Sentiment Analysis

```
% Clear variables
clear; clc;

% Define input sentence
text = "The movie was fantastic and the visuals were breathtaking.";

% Define word lists
positiveWords = ["good", "great", "excellent", "fantastic", "amazing", "breathtaking"];
negativeWords = ["bad", "terrible", "awful", "boring", "worst", "dull"];

% Tokenize text
```

```
words = lower(strsplit(text));

% Count positive and negative words
posCount = sum(ismember(words, positiveWords));
negCount = sum(ismember(words, negativeWords));

% Determine sentiment
if posCount > negCount
   sentiment = "Positive";
elseif negCount > posCount
   sentiment = "Negative";
else
   sentiment = "Neutral";
end

% Display result
disp("Sentiment:");
disp(sentiment);
```

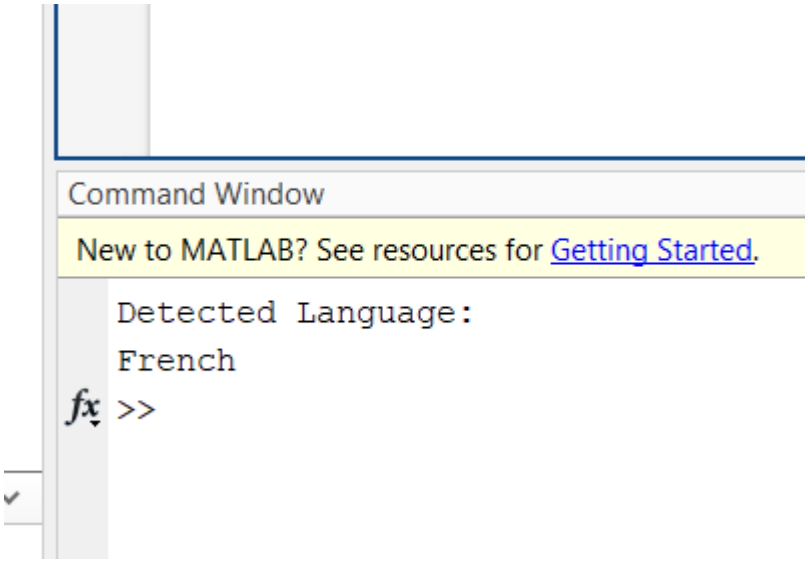


Code: 8

Language Detection (Heuristic)

```
% Clear all
clear; clc;

% Input sentence
text = "Bonjour, comment allez-vous aujourd'hui?";

% Define language-specific keywords
english = ["the", "and", "hello", "good"];
french = ["bonjour", "comment", "vous", "aujourd'hui"];
german = ["hallo", "wie", "geht", "ihnen"];

% Tokenize
words = lower(strsplit(text));

% Count matching words
enCount = sum(ismember(words, english));
frCount = sum(ismember(words, french));
deCount = sum(ismember(words, german));
```

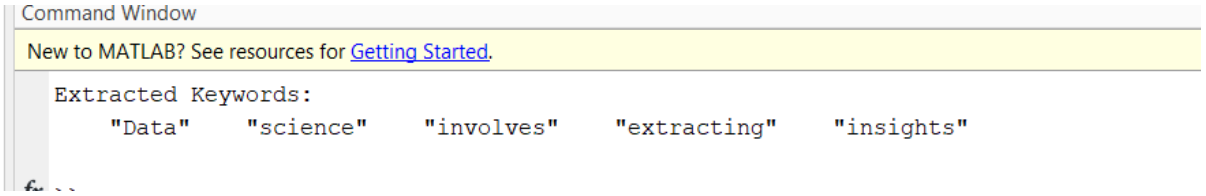
```
% Detect language
[~, idx] = max([enCount, frCount, deCount]);
languages = ["English", "French", "German"];
disp("Detected Language:");
disp(languages(idx));
```



Code: 9

Keyword Extraction from a Paragraph

```
% Clear everything
clear; clc;

% Sample paragraph
text = "Data science involves extracting insights from structured and unstructured data using statistical and machine learning techniques.";

% Tokenize and preprocess
document = tokenizedDocument(text);
document = removeStopWords(document);
document = erasePunctuation(document);

% Bag of words model
bow = bagOfWords(document);

% Sort words by frequency
[sortedCounts, sortedIndex] = sort(bow.Counts, 'descend');
sortedWords = bow.Vocabulary(sortedIndex);

% Extract top 5 keywords
keywords = sortedWords(1:min(5, end));

% Display
disp("Extracted Keywords:");
disp(keywords);
```


Command Window
New to MATLAB? See resources for Getting Started.
Extracted Keywords:
"Data" "science" "involves" "extracting" "insights"